CSE 150: Introduction to Computer Networks

Chris Parsa

Network Layer part 2

1

Chapter 4 Practice Problems

8th edition:

- Chapter 4 R problems: R1, R3, R9, R12, R18, R20, R24, R25, R27, R29 and others as needed
- P problems: P8 and P11
- Will publish more problems soon and some will be done in Group Assignment 6 (publishing that tonight).

4

Chapters 4 and 5: Reading Assignment

Chapter 4:

4.1 Overview of Network Layer

4.2 What's Inside a Router

- Limit reading to class discussion topics

4.3 IP: Internet Protocol

- datagram format
- IPv4 addressing
- NAT
- IPv6

Chapter 5:

5.1 Introduction

5.2 Routing Algorithms

- link state
- distance vector
- hierarchical routing

5.3 OSPF – skim

5.4 BGP

5.6 ICMP

Section numbers from 8th edition of text

9

Chapter 5: Network layer: "control plane" roadmap

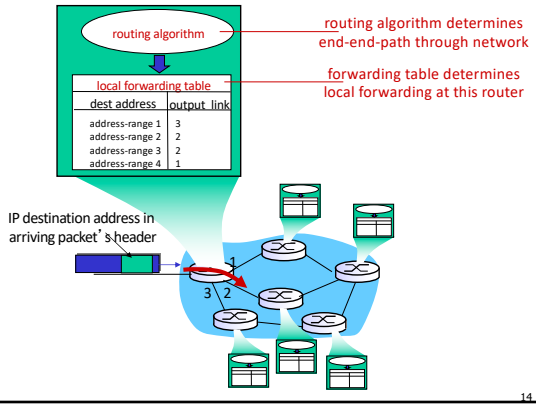
- 5.1 Introduction
- 5.2 routing Algorithms
  - link state
  - distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

- network management, configuration
  - SNMP
  - NETCONF/YANG

Network Layer: 5-13

13

Interplay between routing, forwarding



14

14

Dynamic Routing

Topology and link changes

Goal:

- As network changes occur
  - Determine "good" paths ("routes") from a sending host to a receiving host, through the network core.

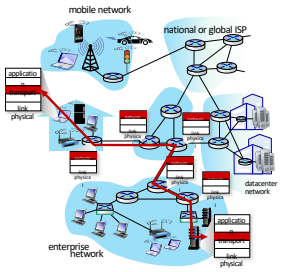
Path: sequence of links packets traverse from a source to final destination

"good" path: AKA least cost path, "fastest", "least congested"

How:

Routing protocol builds forwarding table!
Packets forwarded hop by hop

Routing: a "top-10" networking challenge!



15

Routing algorithm classification

Global/centralized: all routers have *complete* topology, link cost info
- "link state" algorithms

How fast do routes change?

Static: routes change slowly over time

Dynamic: routes change more quickly
- periodic updates or in response to link cost changes

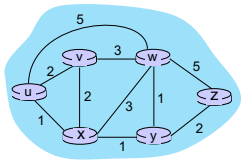
De-centralized: iterative process of computation, exchange of info with neighbors
- routers initially only know link costs to attached neighbors
- "distance vector" algorithms

global or decentralized information?

Two traditional routing algorithms:

- Link State : centralized approach
  - Dijkstra's Algorithm
- Distance Vector: de-centralized approach, rely on neighbor's computation
  - Bellman Ford

Graph abstraction: link costs



graph: $G = (V,E)$
(undirected graph)

$c_{a,b}$: cost of *direct* link connecting *a* and *b*
e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

- Metric: value assigned to link cost
  - Defined by network operator or determined by routing protocol
  - "hop count": cost = 1
  - Bandwidth (inversely related)
  - RTT or metric related to congestion

V: set of routers/subnets = { *u, v, w, x, y, z* }

E: set of physical links ={ *(u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)* }

Neighbors: two nodes with an edge connecting them

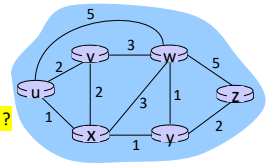
The Cost of the Path…

Cost $(v_1 \rightarrow v_n) = c_{v1,v2} + c_{v2,v3} + \ldots + c_{v(n-1),v(n)}$

1. Least Cost Path

The path between a Source and Destination with the minimum cost

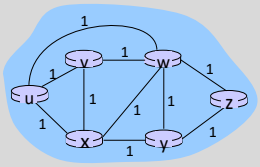
- What's the least-cost path between u and w ?
- (17 paths!!) How determined?



2. Shortest Path – hop count

The path between a with the minimum number of hops

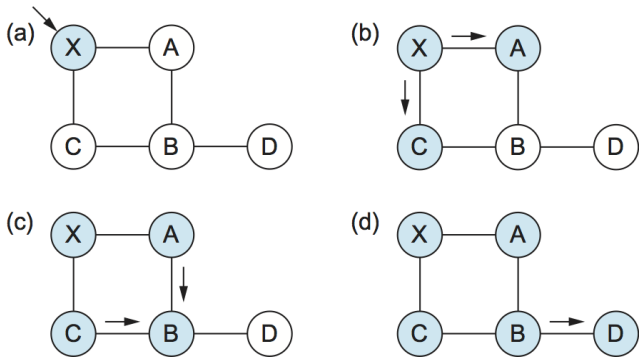
- Cost=1 for all edges
- least cost path = shortest path



Routing Hierarchy

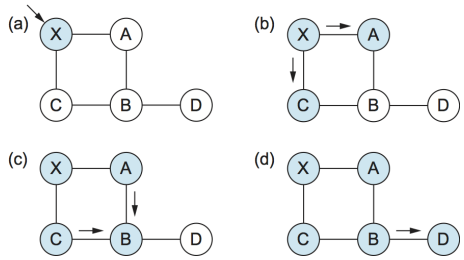
Link State Routing

Flooding of Link State Packets



24

- Which nodes receive X's update?
- How many updates in total does each node receive?
- What is in an update? Size of update?



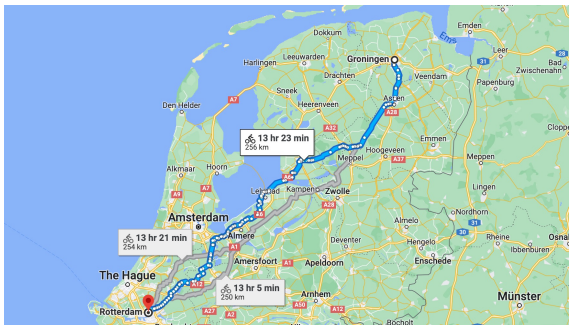
25

After the flood…

- Each node/router/subnet:
  - Knows the complete topology
  - Has the same view of the network

26

Rotterdam -> Groningen



CSE 150 Fall'22

27

Dijkstra's Algorithm

What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city. It is the algorithm for the shortest path, **which I designed in about twenty minutes.**

One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a twenty-minute invention. In fact, it was published in 1959, three years later. The publication is still readable, it is, in fact, quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities. Eventually, that algorithm became to my great amazement, one of the cornerstones of my fame.

— Edsger Dijkstra, in an interview with Philip L. Frana, Communications of the ACM, 2001

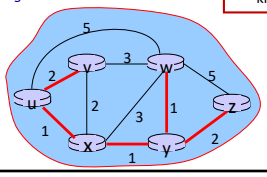
28

Dijkstra's link-state routing algorithm

- Centralized: Topology and link costs to *each* node in network are known
  - "link state broadcast"
  - All nodes have same info
- Iterative: after *k* iterations, the least cost path from a single source node to *k* destinations is known
- Shortest Path Tree: least cost paths from single source node to all destinations
  - produces *forwarding table* for that node

Notation:

- $c_{u,v}$: neighbors' link cost from node *u* to *v*; if not neighbors $c_{u,v} = \infty$
- $D(v)$: *current* cost estimate of least-cost-path from source to destination v
- $p(v)$: predecessor of node v along path from source to *v*
- N': set of nodes whose least-cost-path *definitively* known



Shortest Path Tree rooted at U

Shortest paths from U to all nodes in network

29

Dijkstra's link-state routing algorithm

compute shortest path tree from u to all nodes (*w*)

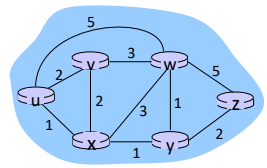
```
Initialization:                 //Loop through u's neighbors, set initial cost
  N' = {u}
  for all nodes v
    if v adjacent to u          //Neighbors only…
      then D(v) = c_{u,v}       //least cost estimate at this point
    else D(v) = ∞               //cost to non-neighbor = ∞

Loop                            //One iteration for each node in network

  find w not in N' such that D(w) is a minimum
  add w to N'                   //"new" node (not in N') with minimum D(w)
  //check w's neighbors:
  update D(v) for all neighbors v of w that aren't in N'
      D(v) = min ( D(v), D(w) + c_{w,v} )
  // update cost of path to neighbors v = min of 1) old D(v)
                                          2) new D(v): D(w) +link cost c_{w,v}
until all nodes are in N'
```

30

Dijkstra's algorithm: computation at node "u"

Step 0 Initialization: $D(v) = c_{u,v}$ for all directly connected neighbors and $c_{u,v} = \infty$ for all non-neighbors. Note predecessor as well.

Step	N'	V D(v),p(v)	W D(w),p(w)	X D(x),p(x)	Y D(y),p(y)	Z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1						
2						
3						
4						
5						



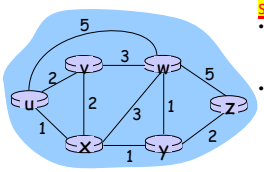
Notation:

- $c_{x,y}$: link cost from node *x* to *y*; $c_{x,y} = \infty$ if not direct neighbors
- *D(v): current* estimate of cost of least-cost-path from source to destination *v*
- *p(v):* predecessor node along path from source to *v*
- *N':* set of nodes whose least-cost-path is *definitively* known

31

Dijkstra's algorithm: computation at node "u"

Step	N'	V D(v),p(v)	W D(w),p(w)	X D(x),p(x)	Y D(y),p(y)	Z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	u,x	2,u	4,x		2,x	
2						
3						
4						
5						



Step 0: Initialization: directly connected neighbors of u.

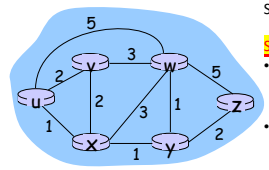
Step 1:

- Pick node with least cost path → choose node x and add node *x* to N';
- For all neighbors of *x* not yet in N' → Re-compute D(v) (look at *v, w, y*) to determine if lower cost
  - e.g., → update D(w): 5 →4 and add D(y)

32

Dijkstra's algorithm: example

Step	N'	V D(v),p(v)	W D(w),p(w)	X D(x),p(x)	Y D(y),p(y)	Z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	u,x	2,u	4,x		2,x	
2	u,x,y		3,y			4,y
3						
4						
5						



Step 0: Initialization: directly connected neighbors of u.
Step 1: Pick node with least cost (x) ; add that to N';
re-compute costs for neighbors not yet in N' (v, w, y)

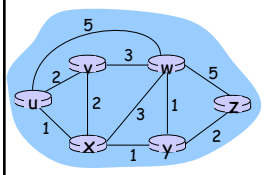
Step 2:

- Pick node with minimum D(v) → Tie between v and y; Choose y arbitrarily (or a rule) and add to N';
- For all neighbors of y net yet in N' →Re-compute D(v) (look at *w, z*)
  → D(w): 4→3 and add D(z)

33

Dijkstra's algorithm: example

Step	N'	V D(v),p(v)	W D(w),p(w)	X D(x),p(x)	Y D(y),p(y)	Z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					
5	uxyvwz					



Step 0: Initialization: directly connected neighbors of u.
Step 1: Pick node with least cost (x) ; add that to N';
re-compute costs for neighbors not yet in N' (v, w, y).
Step 2: Tie between v and y; pick y arbitrarily;
cost of neighbors not yet in N' updated.

Step 3: v is the node with min D(v); add it to N'; check neighbors; w does not need to be updated

Step 4: w is the node with min D(v); add it to N'; check neighbors not yet in N' (z). No update.

Step 5: z is node with min D(v); add it to N'

(Done!)

34

Dijkstra's algorithm: example

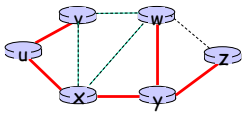
Step	N'	V D(v),p(v)	W D(w),p(w)	X D(x),p(x)	Y D(y),p(y)	Z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			
4	uxyvw					4,y
5	uxyvwz					

Forwarding table in u:

Destination	Next Hop	Cost of Path
v	(u,v)	
x	(u,x)	
y	(u,x)	
w	(u,x)	
z	(u,x)	

Shortest path tree from u:

Shortest path determined by tracing predecessor nodes



35

Practice!! computation at node "u"

Break ties in alphabetical order

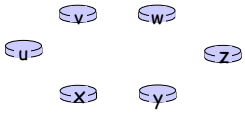
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
		v	w	x	y	z
0						
1						
2						
3						
4						
5						

Forwarding table in u:

Destination	Next Hop	Cost of Path
v		
x		
y		
w		
z		

Shortest path tree from u:

Shortest path determined by tracing predecessor nodes



36

Internet approach to scalable routing

Autonomous systems (AS) (a.k.a. "domains") -- aggregate routers into regions

- e.g., an ISP or a university network

intra-AS (aka "intra-domain") routing *within AS ("network")*

- Each AS can choose an intra-domain routing protocol
  - All routers within AS run same intra-domain protocol

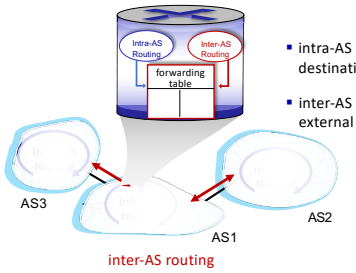
inter-AS (aka "inter-domain") routing *between* ASes

- BGP (Border Gateway Protocol)
- Gateway router: at "edge" of AS, has link(s) to router(s) in other Autonomous Systems
- Gateway routers perform 2 tasks
  - inter-domain routing
  - intra-domain routing

40

Interconnected ASes

Forwarding table configured by intra- and inter-AS routing algorithms



- intra-AS routing -- entries for destinations within AS
- inter-AS & intra-AS -- entries for external destinations

- Link State and Distance Vector are algorithms for intra-domain routing

Network Layer: 5-41

41

Two approaches to finding least cost path:

Link State

- Global information
- Link State Updates flooded: what's in LSU? Who receives them?
- Algorithm?
  - Centralized algorithm – local computation
  - ✓ What is known when it converges?

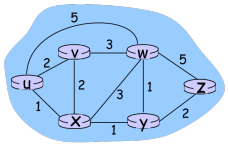
Distance Vector

- Rely on neighbor's computation to determine least cost to all destinations and next hop on path
- Algorithm
  - Bellman-Ford: distributed computation

42

Distance Vector algorithm

Bellman-Ford



Let

$D_x(y)$ = estimate of cost of least-cost path from *x* to *y*

then

$$D_x(y) = \min_v \{c_{x,v} + D_v(y)\} \quad \textit{for each node } y \in N$$

Reported cost of distance from v to destination y

Link cost to neighbor v

min taken over all neighbors v of x

Cost of least cost path from x to y

43

The Setup – rely on neighbors!

45

Bellman Ford algorithm: u→z

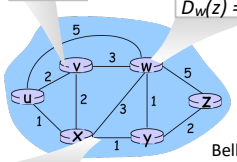
What is the least cost from U to Z?

$D_x(y) \leftarrow \min_v\{c(x,v) + D_v(y)\}$ for each node $y \in N$

all of x's neighbors v

$D_v(z) = 5$

$D_w(z) = 3$



Cost to U's neighbor		Neighbor's Reported Distance
c(u,v)=	+	$D_v(z)$= 5
c(u,w)=		$D_w(z)$= 3
c(u,x)=		$D_x(z)$= 3

$D_x(z) = 3$

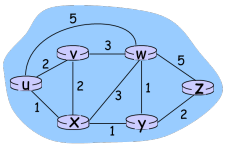
Bellman-Ford equation says:

$D_u(z) = \min($ $)$

- Which nodes are u's neighbors?
- What does result tell us? Cost ($D_u(z)$)of least cost path to all destinations and the next hop used to get there. → add to forwarding table
- What is not known?

46

U's Table



Destination	Next Hop	Distance

47

Distance vector algorithm:

Each node:

wait for (change in local link cost or msg from neighbor)

recompute DV estimate using DV received from neighbor

if DV to any destination has changed, *notify* neighbors

Iterative, Asynchronous: each local iteration caused by:

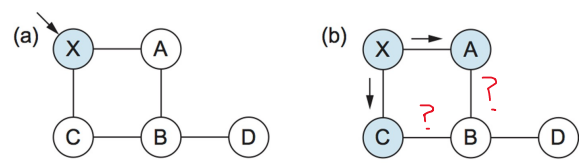
- local link cost change
- DV update received from neighbor
- (or particular protocols, periodic)

Distributed, Self-stopping: Each node notifies neighbors *only* when its DV changes

- Updates *only if necessary*
  - no notification received, no actions taken!

49

Distribution of DV Updates



Type equation here.

$D_x(A)$
$D_x(C)$
$D_x(B)$
$D_x(D)$

X's Distance Vector

- Which nodes receive X's updates?
- How many updates does node C receive?
- What does C do with update?
- Size of the update?

50

DV Implementation: What info does Router X keep?

1. X's Distance Vector:
   - X's distance vector $\mathbf{D}_x = [D_x(y): y \in N]$
     - least cost estimates from x to y ϵ N(all destinations in network)
     - Shared with neighbors
   - next hop to reach destinations
2. Cost to each neighbor *v*: $c_{x,v}$
3. DVs received from neighbors:
   - For each neighbor v: $\mathbf{D}_v = [D_v(y): y \in N]$
     - Used for future calculations as needed

51

Example: Initialize Distance Vectors

Node X table

cost to

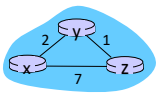
from	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

Initial Distance Vectors

Node Y table

cost to

from	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞



Node Z table

cost to

from	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

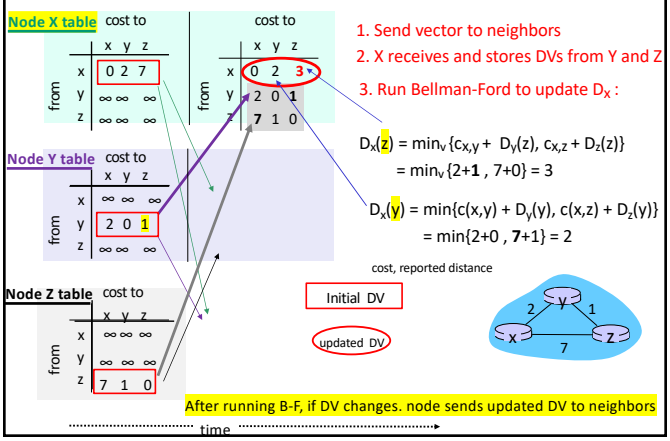
Initial DV

All nodes send their Distance Vector to neighbors

time

52

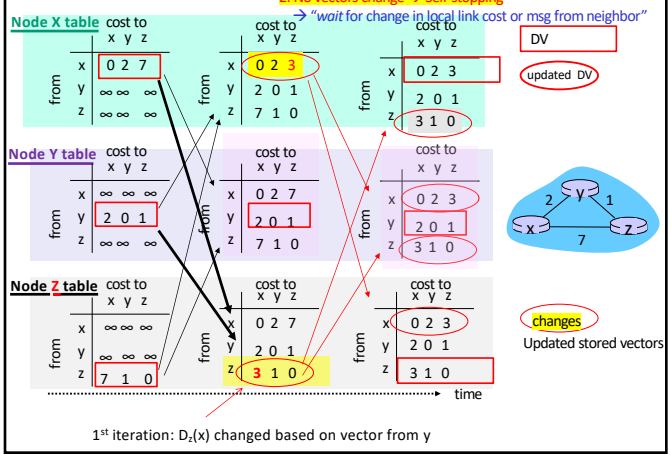
1st iteration



53

2nd Iteration:

1.Only X and Z send their vectors (why not Y?)
2. No vectors change → Self-stopping
→ "*wait* for change in local link cost or msg from neighbor"



1st iteration: $D_z(x)$ changed based on vector from y

54

Comparison of LS and DV algorithms

Message Complexity:

- LS: Each nodes floods its LSU on entire network. All N nodes do this. $O(N^2)$ messages sent
- DV: exchange between neighbors only, time varies
  - Only send when DV changes or periodically according to protocol

Algorithm Convergence

- LS
  - Updates propagate to all nodes N
  - Dijkstra $O(N^2)$
- DV: convergence time varies
  - May be routing loops
  - Count-to-infinity problem

Global vs Centralized:

- LS is a "distributed database" problem → global knowledge
  - LS updates flood link cost over neighbors' direct link, eventually all nodes have same network view
- DV is a "distributed computation" problem
  - DV updates propagate one hop only
  - DV updates carry distance (cost) to **all** known destinations (based on neighbor's computation)
- Trade-off: Flooding smaller packets vs neighbor update of all vectors

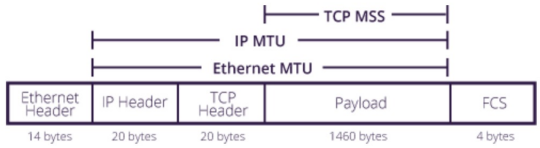
55

Study Sheet: Least Cost Routing Algorithms

- Dijkstra
- Bellman-Ford
- Practice with both examples!
  - Slides
  - Sample exams
- Think about quiet network and when network changes … what happens?
- Understand differences/approach
- Goal of each approach
- Sit back and "think" about each for a few minutes …
  - What is exchanged?
  - What is known at a node after info is exchanged and algorithm is run?

59

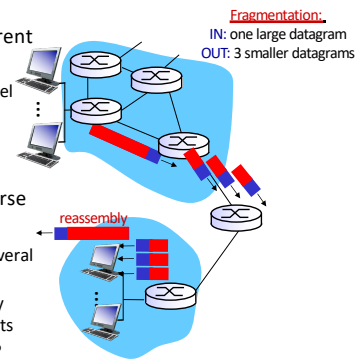
MSS: Maximum Segment Size
MTU: Maximum Transmission Unit



65

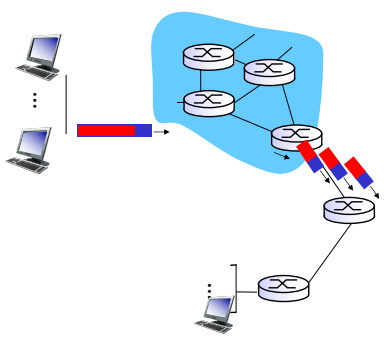
IP fragmentation, reassembly

- Network of Networks: different technologies and links
  - MTU: largest possible link-level frame for a given link
  - MTU can change on path
- Large IP datagrams can be "fragmented" as they traverse network
  - one datagram broken into several smaller datagrams
  - IP header bits used to identify and re-order related fragments
  - Where is "reassembly" done?



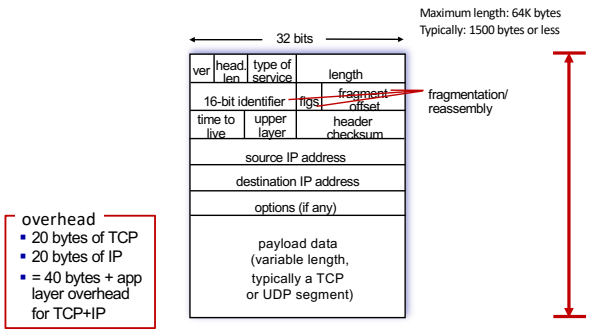
66

1 datagram in … fragments come out



67

IP Datagram Format

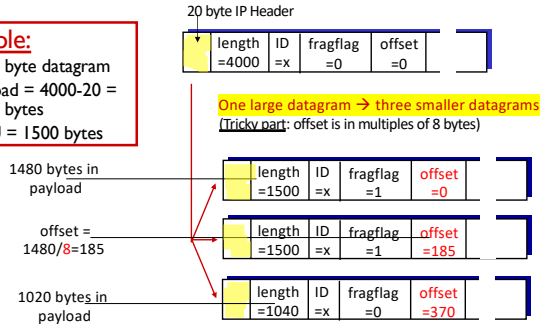


overhead
- 20 bytes of TCP
- 20 bytes of IP
- = 40 bytes + app layer overhead for TCP+IP

68

IP fragmentation

Example:
- 4000 byte datagram
- Payload = 4000-20 = 3980 bytes
- MTU = 1500 bytes



3 fragments: 2x 1480 byte payload
1 x 1020 byte payload

69

NAT: Network Address Translation

solution to address depletion problem?

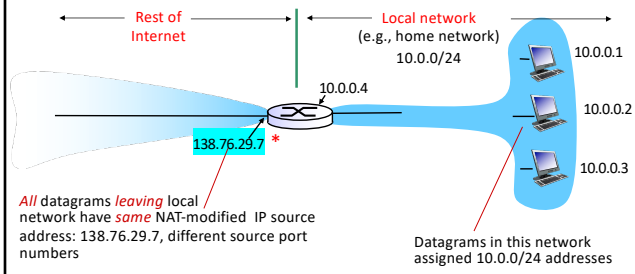
71

NAT: Network Address Translation

- Motivation: 4 billion IP Addresses isn't enough!
- With NAT, all devices in local network assigned 32-bit private IP address (10/8, 172.16/12, 192.168/16 prefixes)
  - only be used locally
- Benefits:
  - Provider ISP supplies only one IP address
  - Can change ISP provider without re-configuring devices in local network
    - DHCP and NAT
  - Security: devices inside local network not directly addressable, visible by outside world
    - local network maintains some privacy

72

NAT: network address translation



Private Address Ranges:
10.0.0.0 /8
172.16.0.0 /20
192.168.0.0 /16

73

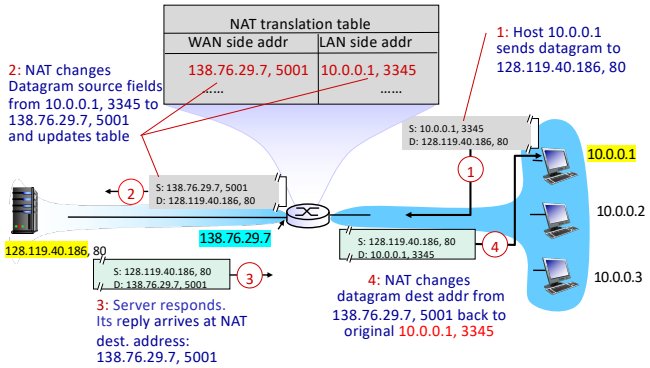
NAT: Network Address Translation

Implementation: NAT router must (transparently):

- Outgoing datagrams: *replace* (source IP, port) → (public IP, new port)
  - remote clients/servers will respond to this IP address and port
- Record in translation table: every translation pair (source IP, port) to (public IP, new port)
  - i.e., everything NAT router has modified
- Incoming datagrams: *replace* incoming packet's destination fields (public IP, port) with actual (IP address, port) stored in NAT table

74

NAT: An Example



76

NAT: Network Address Translation

- 16-bit port-number field:
  - >60,000 simultaneous connections with a single LAN-side address!
  - Performance?
- NAT has been controversial
  - routers "should" only process layer 3
  - violates end-to-end argument (port # manipulation by network-layer device)
  - NAT traversal: what if client wants to connect to server behind NAT?
- NAT is here to stay:
  - extensively used in home and institutional nets, 4G/5G cellular nets
- Address "shortage" should be solved by IPv6
  - 128-bit IP Address

80

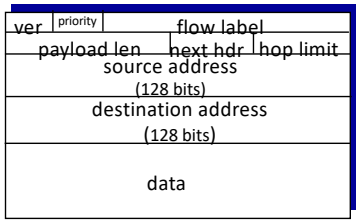
IPv6: motivation

- *Initial motivation:* 32-bit address space soon to be completely allocated
- Solutions:
  - NAT not "nice"
  - Larger IP Address?
- Additional motivation:
  - header format helps speed processing/forwarding
  - header changes to facilitate QoS

82

IPv6 datagram format

- *Fixed-length 40 byte header* (double size of IPv4)
  - IPv6 address is 128 bits! (16 bytes)
- *Priority:* identify priority among datagrams in flow
- *Flow Label:* identify datagrams in same "flow." ("flow" not well defined)
- *Next header:* identify upper layer protocol (TCP, UDP etc) or optional extension headers
- *Hop Limit:* TTL



83

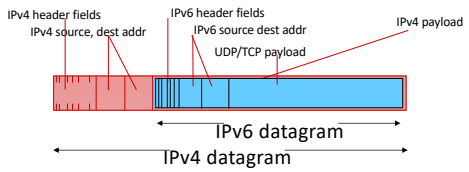
What's missing compared to IPv4?

- *Checksum*: removed entirely to reduce processing time at each hop
- *Options:* allowed, but outside of header, indicated by "Next Header" field
- No fragmentation by IPv6 routers
  - Packet dropped by router if exceeds MTU
  - MTU info can be conveyed in Next Header
- Common goal?
- *ICMPv6:* new version of ICMP
  - additional message types, e.g. "Packet Too Big"

84

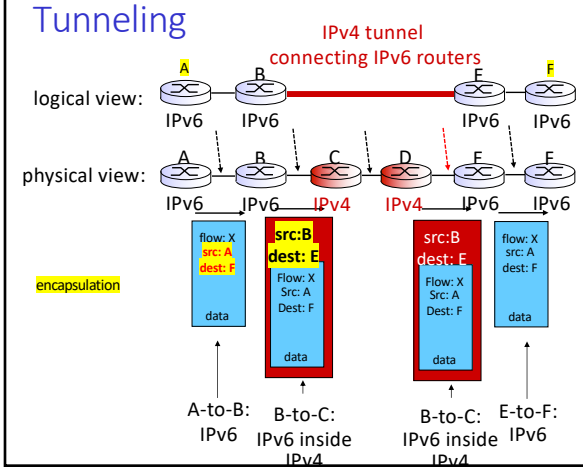
Transition from IPv4 to IPv6

- Not all routers can be upgraded simultaneously
  - no "flag days"
  - How will network operate with mixed IPv4 and IPv6 routers? *
- *Tunneling*: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers



85

Tunneling

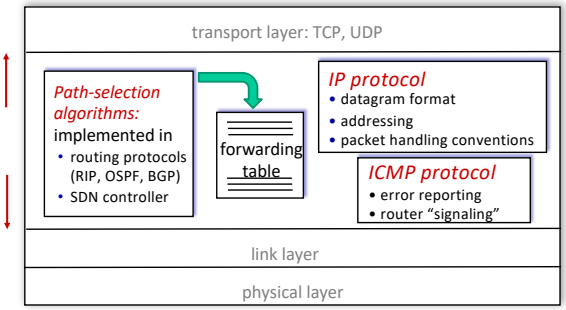


86

IPv6 islands

87

Finishing up Network Layer



88

Make Routing Scalable →Hierarchy

Our routing study thus far - idealized

- all routers identical
- network "flat", ability to know path to all destinations

… not true in practice

Scale: billions of destinations:

- Can't store all destinations in routing tables!
- Forwarding table exchange could swamp links!

Administrative autonomy

- Internet = network of networks
- Divide and conquer with Autonomous System (AS)

89

Autonomous System (AS) decides:

- Intra-domain routing protocol: *e.g.,* LS or DV
- Control over address space within AS
  - Breakdown of address space
  - Reduce size of forwarding table/lookup time
  - Security benefits
- Policy Based Routing
  - Peering Agreements with other Providers
    - Traversal/Cross-traffic: allow or disallow
    - Advertisement of prefixes
    - Cost $$
- Connect to adjacent AS via Border Router at "edge" of AS
- Break down the Internet routing problem into
  - manageable size
  - connect Ases together

90

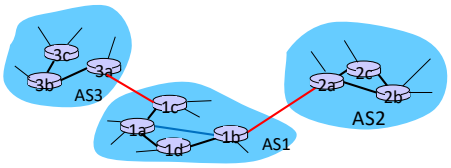
Intra-AS routing: routing within an AS

Most common intra-AS routing protocols:

- RIP: Routing Information Protocol [RFC 1723]
  - Included in BSD-UNIX distribution in 1982
  - classic DV: DVs exchanged every 30 secs
  - no longer widely used, has loops and counting to infinity problem
- EIGRP: Enhanced Interior Gateway Routing Protocol
  - DV based
  - formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868])
- OSPF: Open Shortest Path First [RFC 2328]
  - link-state routing
  - Very common routing protocol
  - IS-IS protocol (ISO standard, not RFC standard) essentially same as OSPF

91

Autonomous Systems



Two types of routing needed:

- intra-domain routing
- inter-domain routing

ASes are numbered

Focus on Border Gateway Routers

92

Inter/Intra -AS tasks

Suppose router 1a in AS1 must forward a datagram whose destination is outside of AS1:

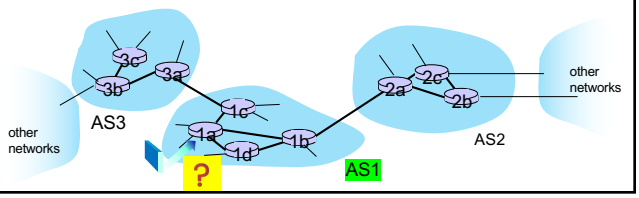
- Router must forward packet to a border router, but which one?
- **Best** way to reach gateway router?

AS1 inter-domain routing :

1. Learn what destinations (prefixes) are reachable through AS2 and AS3
2. Propagate this reachability info to all AS1 routers

Job of intra-AS routing:

Determine least cost path to reach Border Router



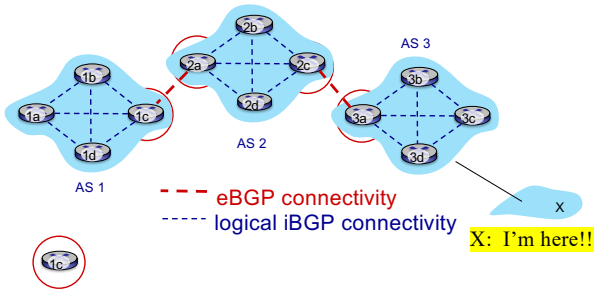
93

Border Gateway Protocol (BGP)

- BGP : *the* de facto inter-domain routing protocol
  - Internet's inter-AS routing protocol
  - "glue that holds the Internet together"
- Allows subnets to advertise their existence, and the destinations they can reach, to rest of Internet: *"I am here, this is who I can reach, and how"*
- Two logical components:
  - eBGP: obtain subnet reachability information from neighboring ASes
  - iBGP: propagate reachability information to AS-internal routers
- "good" routes to other networks based on reachability and policy
  - Note: NOT necessarily least cost!

95

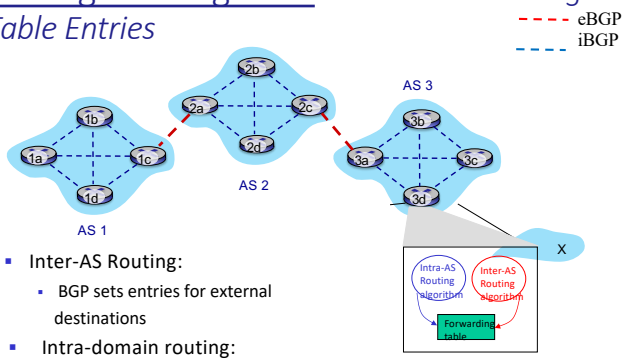
Interconnect AS with eBGP, iBGP



Border gateway routers run both eBGP and iBGP protocols

96

Putting it all together: *Router Forwarding Table Entries*



- Inter-AS Routing:
  - BGP sets entries for external destinations
- Intra-domain routing:
  - OSPF, RIP or another protocol set entries for internal destinations

97

BGP Message Exchange:

- BGP advertised routes: Prefix + Attributes
  - Prefix: destination subnet being advertised
  - Two important Attributes:
    - AS-PATH: list of ASes through which prefix advertisement has passed
    - NEXT-HOP: indicates specific internal-AS router to next-hop AS

98

BGP Policy Base Routing

Often multiple paths to a prefix are known

→ AS Policy dictates propagation of paths

Accept? Gateway receiving advertisement accepts or declines path according to *Import Policy*

- e.g., never route through AS 200

Further advertise? Path propagation is determined by *AS Policy*

99

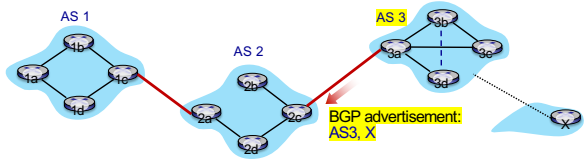
BGP Basics: "path vector" protocol

BGP session: Peers BGP routers exchange messages over semi-permanent TCP connection:

- Prefix + Attributes

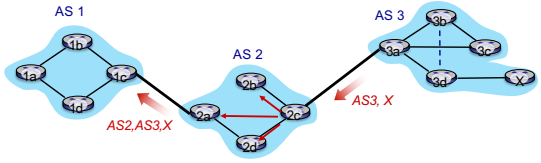
AS3 router 3a advertises path AS3,X to gateway 2c:

- "X is over here!"
- AS3 provides the complete AS path to X
  - *promises* to AS2 it will forward datagrams towards X (and)



100

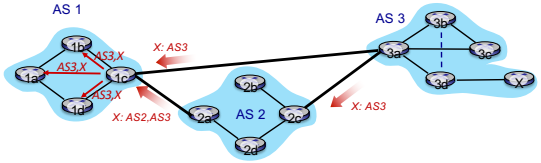
Path advertisement: Who learns about X?



- AS2: router 2c receives path advertisement via eBGP from AS3 router 3a
  - Example advertisement: X: **AS-PATH**: AS3; **NEXT-HOP**: 3a
- Based on AS2 policy: 2c accepts path X: AS3 and uses iBGP to propagate to all AS2 routers
- Based on AS2 policy: router 2a uses eBGP to advertise:
  - X: AS-PATH AS2, AS3; NEXT-HOP: 2a to AS1 router 1c
  - What is implication of this announcement?

101

Which path to choose?



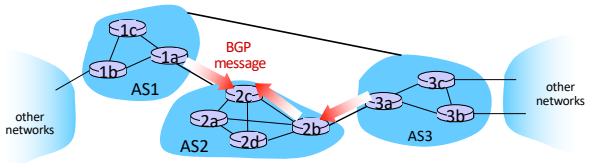
Gateway router may learn multiple paths to destination:

- AS1 gateway router 1c learns path X: *AS2,AS3* from 2a
- AS1 gateway router 1c learns path X: *AS3* from 3a

→Based on *policy,* AS1 router 1c chooses path X: *AS3* and used iBGP to advertise path within AS1 (see selection criteria on next slide)

102

BGP Path Selection Criteria



A router may learn about more than one route to destination AS.

Select route based on:

1. Local preference value attribute: policy decision
2. Shortest AS-PATH
3. Closest NEXT-HOP router
4. Additional criteria – AS decides

Note: AS Policy trumps shortest AS-PATH

106

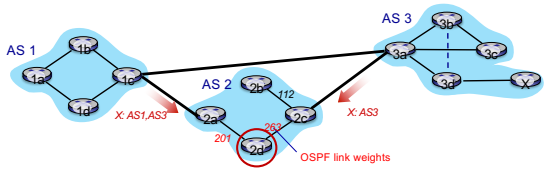
Shortest AS-PATH

Example: select

- AS2 AS17 to 138.16.64/22
- AS3 AS131 AS201 to 138.16.64/22

107

Hot potato routing
--Fastest way out of AS !

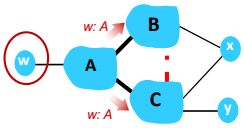


- 2d learns (via iBGP) it can route to X via 2a or 2c
- Hot potato routing:
  - Choose gateway with least *intra-domain* cost (e.g., use OSPF)
  - Choose 2a
  - → result is longer AS PATH to *X!* Hot potato doesn't worry about inter-domain cost!
- BGP Policy trumps both AS PATH and Hot Potato

108

To advertise or not? That is the question!

Typical "real world" policy:
ISP prefers to route traffic to/from its customer networks
(doen't want to carry transit traffic between other ISPs



Legend:

are Provider networks

w,x,y are customer networks

- A advertises path w: A to both B and C
- B *chooses not to advertise w:* BA to C!
  - B gets no "revenue" for routing w: C,B,A since C, A and w is not B's customer
  - B chooses not to advertise w: B,A to C
    - C does *not* learn about the path
- No traversal through B: C will use route w: C,A and traffic doesn't go through B!
  - eBGP: C sends w: CA to y and x

110

Why different Intra-, Inter-AS routing ?

Policy:

- inter-AS: Admin wants control over how its traffic routed, who routes through its network
- intra-AS: single admin, so policy less of an issue, "best route" is important

Scale:

- hierarchical routing reduced table size, reduced update traffic
- Connect AS

Performance:

- intra-AS: can focus on performance
- inter-AS: policy dominates over performance

Reading Assignment: Section 5.4

113